

Architecture

CO3 G10

Will Barlow
Harris Byrom
Corbin Hedger
Dmytro Kovalenko
Sharjil Munshi
Oliver Spence
Joshua Thomas
Leon Varghese

Languages and Tools

We first started by identifying some potential platforms that we could use to create the UML diagrams:

- Lucidchart
- Canva
- Smartdraw
- PlantUML

After some consideration we decided to use the PlantUML plugin in IntelliJ to create the UML diagrams. This is because PlantUML is easy to understand and is entirely text based which means that we do not need to use any complex graphical editors. It also supports all of the diagram types that we need to create and automatically handles the layout of these diagrams, making the process simple. The IntelliJ IDE allows us to instantly preview the diagrams while we are creating them which helps with troubleshooting.

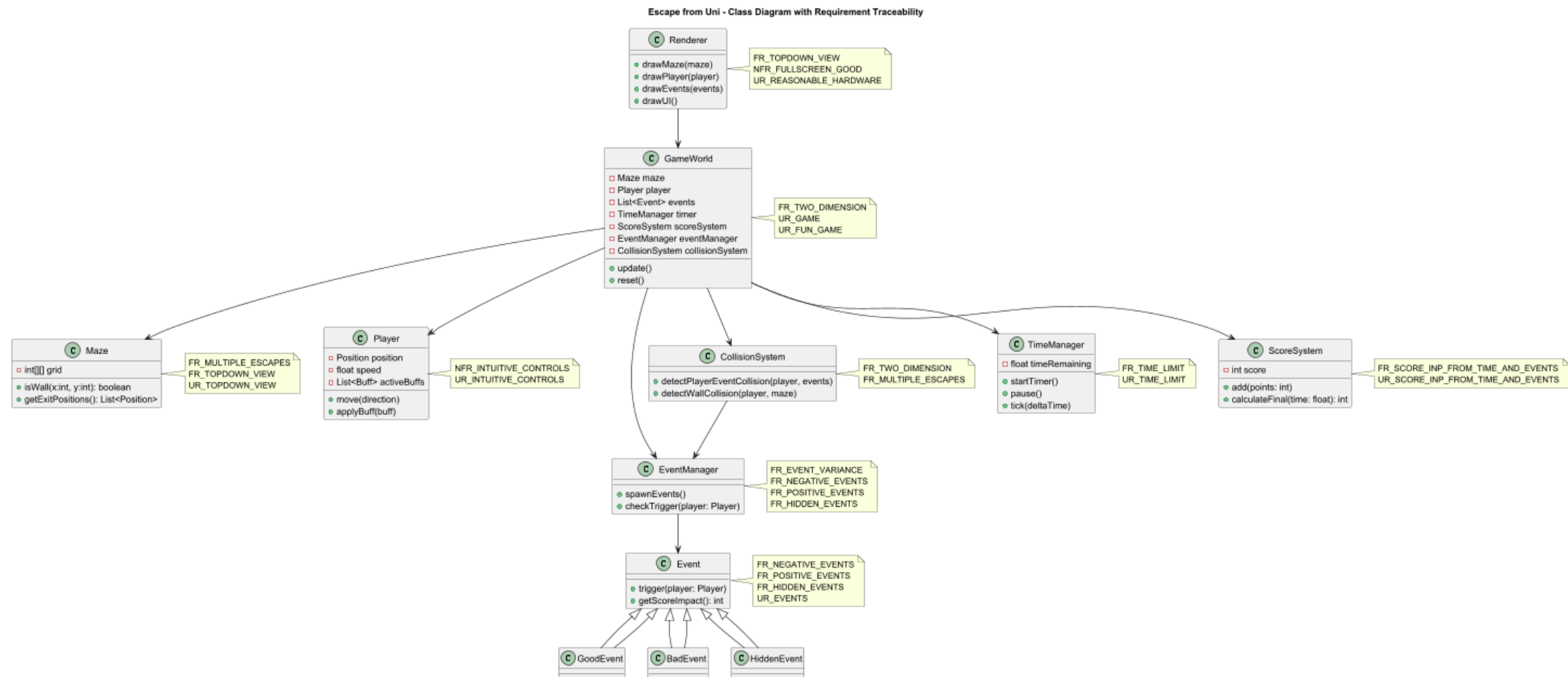
Design Process and Evolution of Architecture

Interim versions of UML diagrams which were used in the early stages of development can be found on the website. The initial component diagrams started with all functionality embedded in a single class and then developed to implement layered architecture allowing for future expansion. The initial class diagram consisted of simple structure with no inheritance for events yet and no external systems such as score or timer. The initial sequence diagrams started with basic movement and collision handling before evolving to introduce event logic, however it still lacks scoring and visual logic. The Initial state diagram demonstrates a simple start-stop flow with no menus, timing, or pause considered. The final versions of each UML diagram are built upon these prototypes and contain full detail. Additionally they relate to our requirements with the added references which are noted in yellow boxes which gives requirement traceability.

In addition to the UML diagrams, we used CRC cards to help design the structure of the code. We decided to use a delegated control style, as it specializes objects to the job they need to fulfill, while also making sure that code can be reused for similar objects, and it is easier to maintain and scale than a centralized style. The CRC cards also helped inform our UML diagrams, as we knew what level of centralization we should design our diagrams around, and finally it helped us choose to structure the code in an event-based model, where the gameloop takes actions based on certain events happening.

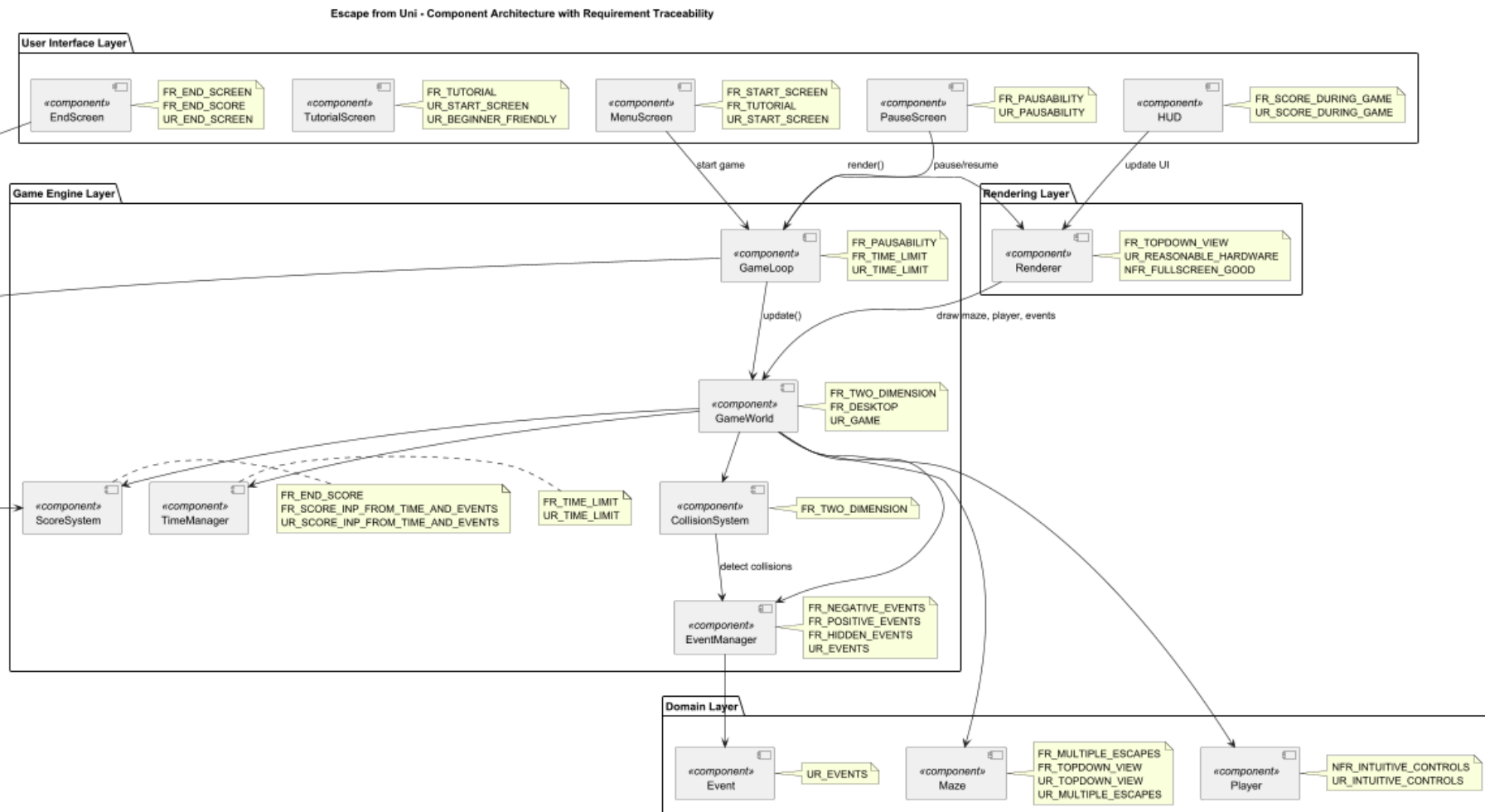
Class Diagram with Requirement Traceability

The class diagram below shows the functions that are within each of the classes, along with the requirements that were fulfilled as a result of those classes (which can be seen in yellow). It demonstrates modularity and separation of major components within the program and the link between each class in the hierarchy.



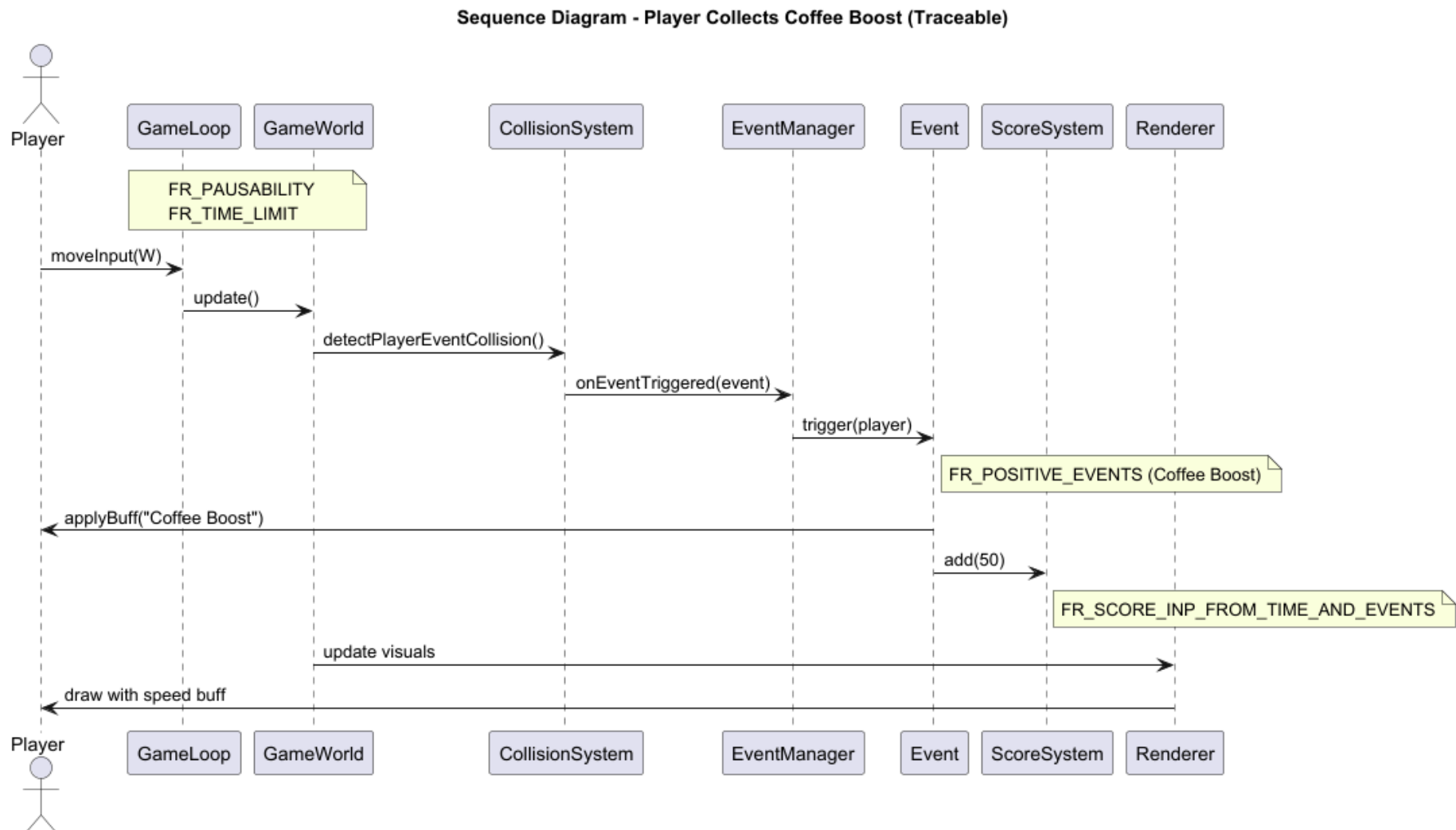
Component Architecture with Requirement Traceability

The diagram below shows the different layers of the program. The user interface layer refers to all the interface related features (along with all the interface related requirements fulfilled (shown in yellow)). The game engine layer refers to all the game engine related features such as scoring, time, collision, event, game loop etc. The input layer handles the user related controls and the rendering layer ensures that the rendering related requirements are fulfilled. The links between the different layers illustrated using labelled arrows.



Player Collect Coffee Boost Sequence Diagram

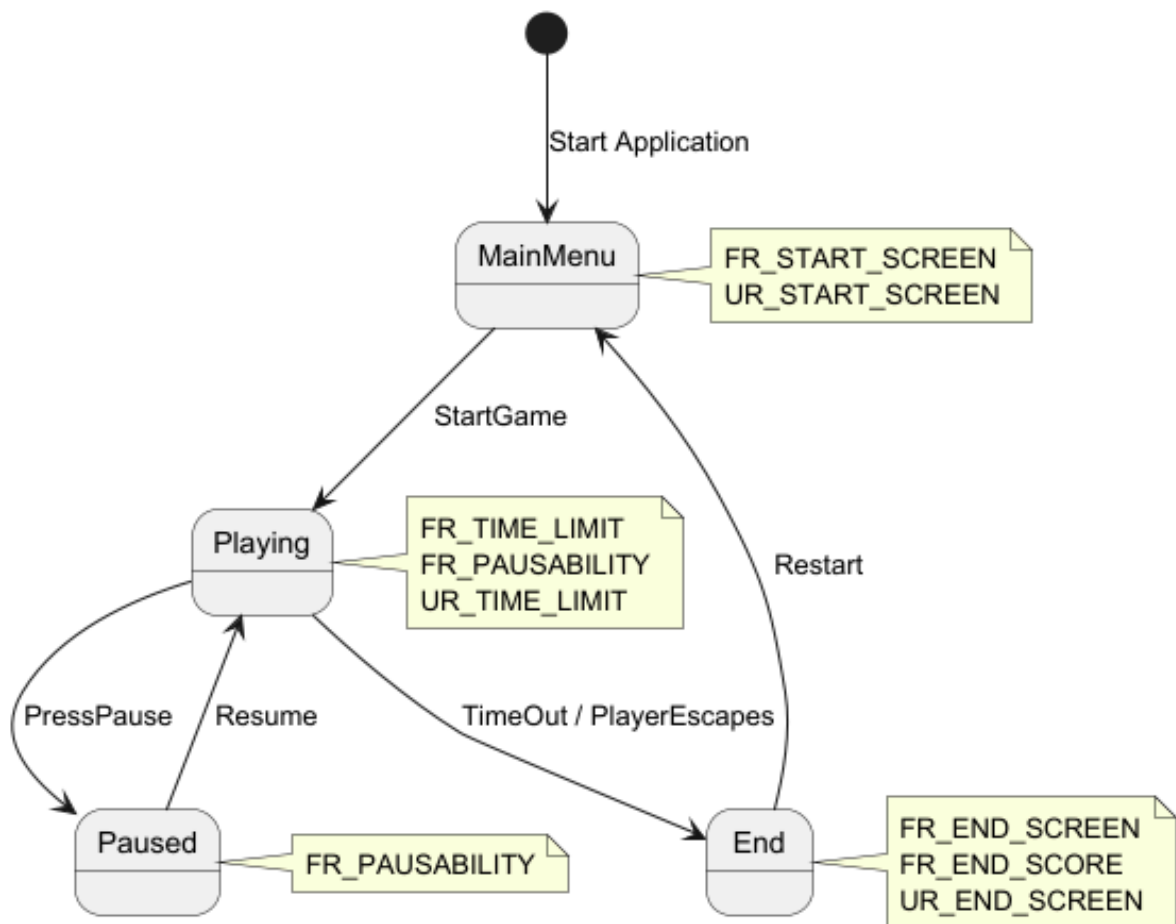
The diagram below shows the sequence of the relevant functions/classes for the coffee boost feature. The sequence diagram helped us to design this feature as it gave us an outline of which functions could be programmed simultaneously and which ones needed to be programmed first (due to dependencies). E.g in our case, the `onEventTriggered(event)` function was dependent on the `detectedPlayerEventCollision()` with the event being passed as an input into the function. This diagram helped guide us to plan and execute our functions properly. The labels in yellow state the requirements that these functions helped to fulfil.



Game State Machine

This diagram shows the flow of the game states, and defines specific behavior and rules for transitions between states. This is important as it provides the structure to move between menus, such as the start and end screens (FR_START_SCREEN and FR_END_SCREEN) and then back to playing the game again, as well as defining what happens when the game is paused (FR_PAUSABILITY). We implemented this as the GameStateHandler, which changes GameState and is used by menus to pause and restart the gameplay.

Game State Machine - Escape from Uni (with Traceability)



Game Loop Diagram

This diagram shows a high level representation of the gameloop that the game follows, from handling inputs to passing them through game logic to apply the effect on the maze entities. We chose to do this so that the processes applied are all encapsulated within the GameLogic (implemented as Game), which helps separate the processing of inputs and outputs so that they can easily be handled, whether it be audio (FR_MUSIC_OR_SOUND) or visual updates. The diagram also shows how GameLogic handles the game being paused (FR_PAUSABILITY), as it uses GameStateHandler to check and change the state, and if the game is paused, keyboard inputs are not used until the user unpauses the game.

